

Laporan Praktikum Pekan 3

“LOGISTIC REGRESSION”

Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

AFDHAL DINILHAK S.Kom, M.Kom



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026**

Daftar Isi

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum.....	3
LINK GITHUB:.....	4
BAB II LANGKAH-LANGKAH PRAKTIKUM	4
Latihan 1 –Logistic Regression	4
2.1 Import Library.....	4
2.2 Load dan Menampilkan Dataset	4
2.3 Visualisasi Distribusi Data Menggunakan Histogram	5
2.4 Korelasi Visualisasi Matriks Korelasi.....	6
2.5 Visualisasi Scatter Plot.....	7
2.6 Selecting Features	8
2.7 Splitting Data	8
8. Pelatihan Model Logistic Regression	9
9. Confusion Matrix	9
10. Visualisasi Confusion Matrix.....	10
11. Classification Report.....	11
12. ROC Curve dan AUC Score	11
TUGAS PRAKTIKUM:	12
1. Scaling Dataset.....	12
2. Membagi Data Menjadi Data Latih dan Data Uji	15
3. Fungsi Evaluasi Model dengan Threshold.....	15
4. Logistic Regression Tanpa Scaling	16
5. Logistic Regression Dengan Scaling	17
6. Perbandingan Hasil Model.....	18
7. ROC Curve dan AUC Score	19
8. Analisis Hasil.....	20
BAB III PENUTUP	21
3.1 Kesimpulan	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Machine Learning merupakan salah satu cabang dari Artificial Intelligence (AI) yang memungkinkan komputer untuk mempelajari pola dari data dan membuat prediksi tanpa harus diprogram secara eksplisit. Salah satu tugas utama dalam machine learning adalah klasifikasi, yaitu proses mengelompokkan data ke dalam kelas tertentu berdasarkan karakteristik yang dimiliki.

Salah satu algoritma klasifikasi yang cukup populer adalah Logistic Regression. Meskipun memiliki nama “regression”, algoritma ini sebenarnya digunakan untuk melakukan klasifikasi, khususnya binary classification atau klasifikasi dua kelas. Logistic Regression bekerja dengan memanfaatkan fungsi sigmoid untuk menghasilkan nilai probabilitas antara 0 dan 1 yang kemudian digunakan untuk menentukan kelas data.

Pada praktikum ini digunakan dataset Pima Indians Diabetes Database untuk membangun model Logistic Regression dalam memprediksi apakah seseorang menderita diabetes atau tidak berdasarkan beberapa fitur kesehatan seperti Glucose, BMI, BloodPressure, dan lainnya. Selain proses pelatihan model, praktikum ini juga membahas berbagai metode evaluasi seperti Confusion Matrix, Accuracy, Precision, Recall, F1-Score, ROC Curve, dan AUC Score untuk menganalisis performa model klasifikasi.

Melalui praktikum ini diharapkan mahasiswa dapat memahami konsep dasar Logistic Regression, proses klasifikasi data, serta teknik evaluasi model machine learning dalam menyelesaikan permasalahan klasifikasi biner.

1.2 Tujuan Praktikum

1. Memahami perbedaan antara regresi dan klasifikasi.
2. Mengimplementasikan Logistic Regression untuk binary classification.
3. Menghitung dan menginterpretasikan confusion matrix, accuracy, precision, recall, F1-score, ROC Curve, dan AUC score.
4. Memahami pengaruh threshold terhadap performa model.
5. Menganalisis performa model Logistic Regression pada dataset klasifikasi.
6. Menganalisis hasil klasifikasi berdasarkan dataset yang digunakan pada latihan praktikum.

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%203

BAB II

LANGKAH-LANGKAH PRAKTIKUM

Latihan 1 –Logistic Regression

2.1 Import Library

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
```

Penjelasan:

Pada tahap awal dilakukan import beberapa library yang diperlukan untuk proses pengolahan data, visualisasi data, pembangunan model Logistic Regression, dan evaluasi model. Library pandas digunakan untuk manipulasi data berbentuk DataFrame, numpy digunakan untuk komputasi numerik, matplotlib dan seaborn digunakan untuk visualisasi data, sedangkan sklearn digunakan untuk proses pembagian dataset, pelatihan model, dan evaluasi performa model klasifikasi.

2.2 Load dan Menampilkan Dataset

```
dataset_url = 'https://raw.githubusercontent.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/main/PRAKTIKUM%203/diabetes.csv'
df = pd.read_csv(dataset_url)
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Penjelasan:

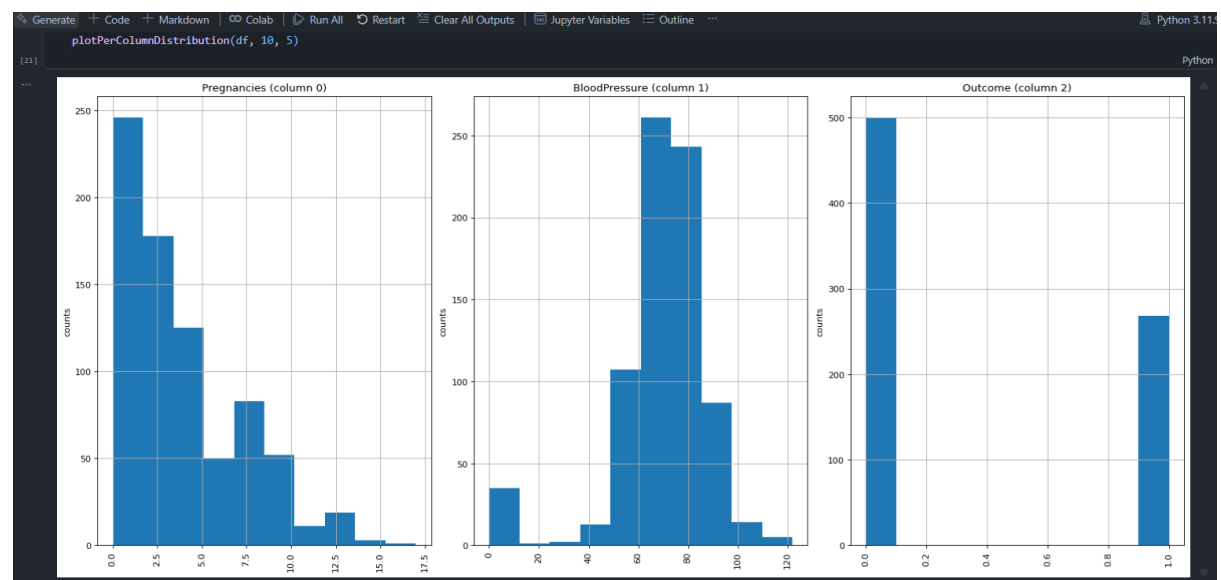
Dataset diabetes dimuat menggunakan fungsi `pd.read_csv()` dengan memanfaatkan URL raw dari repository GitHub. Dataset ini berisi beberapa fitur kesehatan pasien seperti jumlah kehamilan, kadar glukosa, tekanan darah, BMI, dan usia yang digunakan untuk memprediksi apakah pasien menderita diabetes atau tidak.

Fungsi `df.head()` digunakan untuk menampilkan lima data pertama sebagai bentuk validasi bahwa dataset berhasil dimuat dengan benar.

2.3 Visualisasi Distribusi Data Menggunakan Histogram

```
import math

def plotPerColumnDistribution(df, nGraphShown, nGraphPerRow):
    nunique = df.nunique()
    df = df[[col for col in df if nunique[col] > 1 and nunique[col] < 50]]
    nRow, nCol = df.shape
    columnNames = list(df)
    nGraphRow = math.ceil(nCol / nGraphPerRow)
    plt.figure(num=None, figsize=(6 * nGraphPerRow, 8 * nGraphRow), dpi=80, facecolor='w', edgecolor='k')
    for i in range(min(nCol, nGraphShown)):
        plt.subplot(nGraphRow, nGraphPerRow, i + 1)
        columnDf = df.iloc[:, i]
        if (not np.issubdtype(type(columnDf.iloc[0]), np.number)):
            valueCounts = columnDf.value_counts()
            valueCounts.plot.bar()
        else:
            columnDf.hist()
            plt.ylabel('counts')
            plt.xticks(rotation=90)
            plt.title(f'{columnNames[i]} (column {i})')
    plt.tight_layout(pad=1.0, w_pad=1.0, h_pad=1.0)
    plt.show()
```



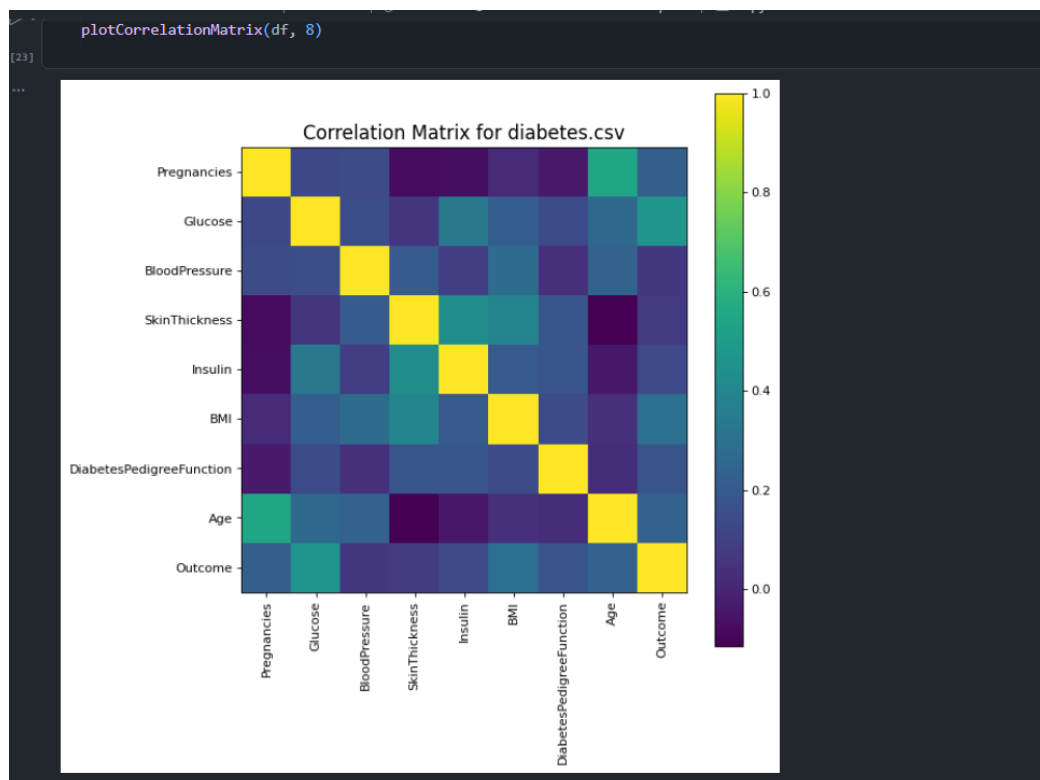
Penjelasan:

Pada tahap ini dilakukan visualisasi distribusi data menggunakan histogram untuk setiap fitur pada dataset. Histogram digunakan untuk melihat pola distribusi data, persebaran nilai, dan kemungkinan adanya data yang tidak seimbang.

Visualisasi ini membantu dalam memahami karakteristik dataset sebelum dilakukan pelatihan model Logistic Regression.

2.4 Korelasi Visualisasi Matriks Korelasi

```
# Correlation matrix
def plotCorrelationMatrix(df, graphWidth):
    filename = "diabetes.csv"
    df = df.dropna(axis='columns') # drop columns with NaN
    df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are more than 1 unique values
    if df.shape[1] < 2:
        print(f'No correlation plots shown: The number of non-NaN or constant columns ({df.shape[1]}) is less than 2')
        return
    corr = df.corr()
    fig, ax = plt.subplots(num=None, figsize=(graphWidth, graphWidth), dpi=80, facecolor='w', edgecolor='k')
    corrMat = ax.matshow(corr)
    plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
    plt.yticks(range(len(corr.columns)), corr.columns)
    plt.gca().xaxis.tick_bottom()
    plt.colorbar(corrMat)
    plt.title(f'Correlation Matrix for {filename}', fontsize=15)
    plt.show()
```



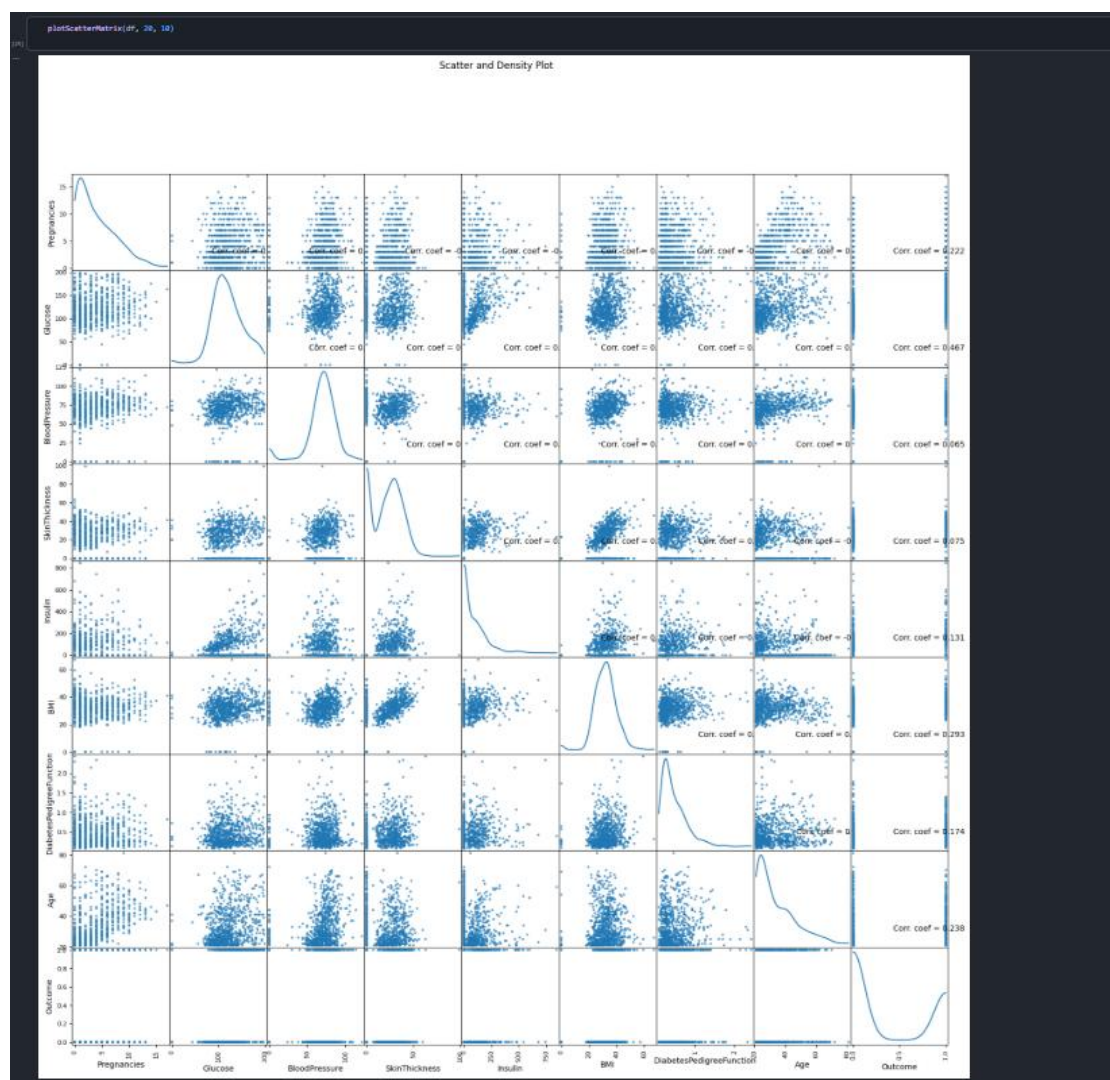
Penjelasan:

Pada tahap ini dilakukan analisis hubungan antar fitur menggunakan matriks korelasi. Korelasi digunakan untuk mengetahui seberapa kuat hubungan antar variabel dalam dataset.

Visualisasi heatmap korelasi membantu dalam mengidentifikasi fitur yang memiliki hubungan kuat terhadap variabel target sehingga dapat digunakan untuk meningkatkan pemahaman terhadap data yang digunakan.

2.5 Visualisasi Scatter Plot

```
# Scatter and density plots
def plotScatterMatrix(df, plotSize, textSize):
    df = df.select_dtypes(include=[np.number]) # keep only numerical columns
    # Remove rows and columns that would lead to df being singular
    df = df.dropna(axis='columns')
    df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are more than 1 unique values
    columnNames = list(df)
    if len(columnNames) > 10: # reduce the number of columns for matrix inversion of kernel density plots
        columnNames = columnNames[:10]
    df = df[columnNames]
    ax = pd.plotting.scatter_matrix(df, alpha=0.75, figsize=[plotSize, plotSize], diagonal='kde')
    corrs = df.corr().values
    for i, j in zip(*plt.np.triu_indices_from(ax, k = 1)):
        ax[i, j].annotate('Corr. coef = %.3f' % corrs[i, j], (0.8, 0.2), xycoords='axes fraction', ha='center', va='center', size=textSize)
    plt.suptitle('Scatter and Density Plot')
    plt.show()
```



Penjelasan:

Scatter plot digunakan untuk melihat pola hubungan dan persebaran data antar fitur dalam dataset. Visualisasi ini membantu dalam mengetahui apakah terdapat pola tertentu atau hubungan linear antar variabel.

Selain itu, scatter plot juga dapat membantu mendeteksi outlier atau data yang menyimpang dari pola umum dataset.

2.6 Selecting Features

```
[26] feature_cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']
      X = df[feature_cols]
      y = df.Outcome

[27] print(X.shape)
      print(y.shape)

... (768, 7)
      (768,)
```

Penjelasan:

Pada tahap ini dataset dipisahkan menjadi dua jenis variabel yaitu variabel independen (fitur) dan variabel dependen (target).

Variabel X digunakan sebagai fitur input model, sedangkan variabel y digunakan sebagai label target yang akan diprediksi oleh model Logistic Regression.

2.7 Splitting Data

```
[28] from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=16)
```

Penjelasan:

Dataset kemudian dibagi menjadi data pelatihan dan data pengujian menggunakan fungsi `train_test_split()`.

Sebanyak 70% data digunakan untuk proses pelatihan model dan 30% digunakan untuk pengujian model. Pembagian data ini bertujuan agar performa model dapat diuji menggunakan data yang belum pernah dilihat sebelumnya.

8. Pelatihan Model Logistic Regression

```
logreg = LogisticRegression(random_state=16)

logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

y_pred
[13] ✓ 0.1s

... array([1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1,
          0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1,
          0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0,
          0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1,
          0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1,
          0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
          0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,
          1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0,
          1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1])
```

Penjelasan:

Pada tahap ini dilakukan proses pelatihan model menggunakan algoritma Logistic Regression. Model diinisialisasi menggunakan fungsi `LogisticRegression()` kemudian dilatih menggunakan data training dengan metode `fit()`.

Setelah proses pelatihan selesai, model digunakan untuk memprediksi data pengujian menggunakan fungsi `predict()`.

9. Confusion Matrix

```
cnf_matrix = confusion_matrix(y_test, y_pred)
cnf_matrix
[14] ✓ 0.0s

... array([[134, 13],
          [ 35, 49]])
```

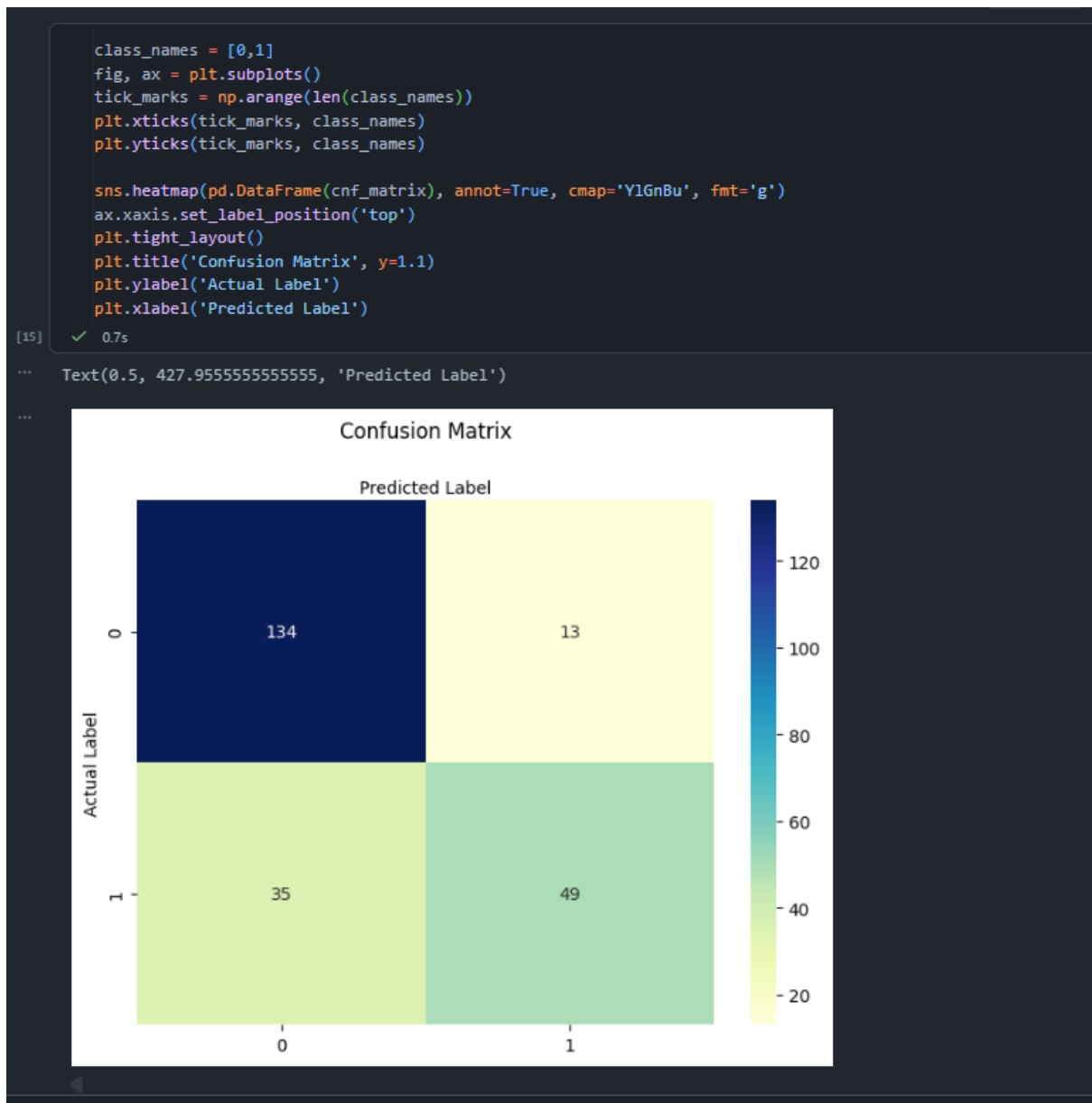
Penjelasan:

Confusion Matrix digunakan untuk mengevaluasi performa model klasifikasi dengan membandingkan hasil prediksi model terhadap label sebenarnya.

Confusion Matrix terdiri dari empat komponen utama yaitu True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Melalui confusion matrix

dapat diketahui jumlah prediksi yang benar maupun kesalahan klasifikasi yang dilakukan model.

10. Visualisasi Confusion Matrix



Penjelasan:

Confusion Matrix divisualisasikan menggunakan heatmap dari library seaborn agar lebih mudah dianalisis.

Visualisasi ini membantu dalam melihat jumlah prediksi benar dan salah secara lebih jelas sehingga performa model dapat dievaluasi dengan lebih baik.

11. Classification Report

```
target_names = ['without diabetes', 'with diabetes']
print(classification_report(y_test, y_pred, target_names=target_names))
```

[16] ✓ 0.1s

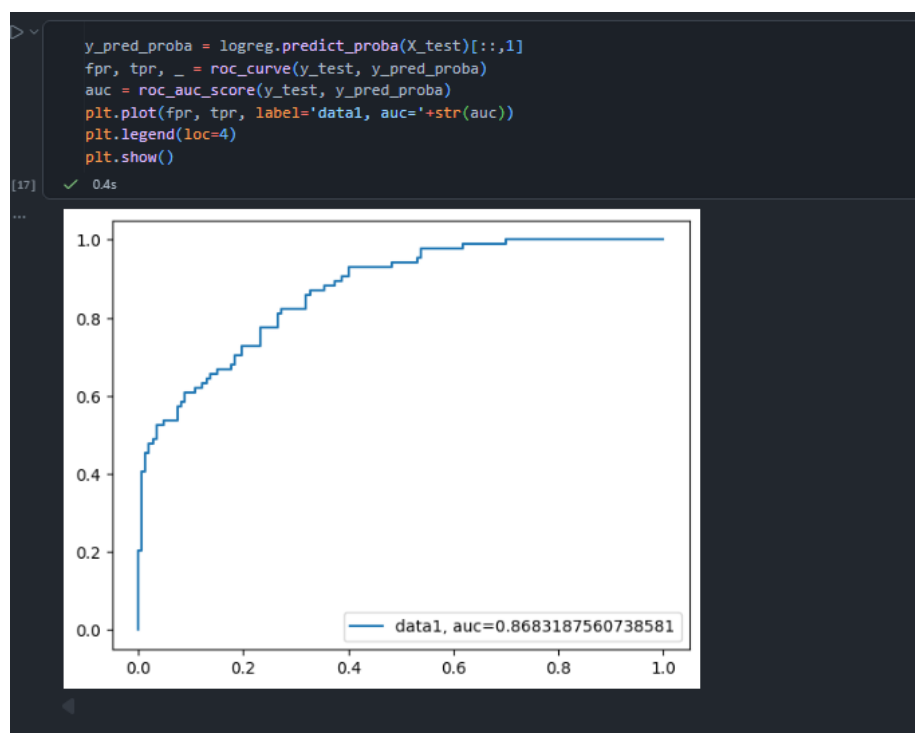
	precision	recall	f1-score	support
without diabetes	0.79	0.91	0.85	147
with diabetes	0.79	0.58	0.67	84
accuracy			0.79	231
macro avg	0.79	0.75	0.76	231
weighted avg	0.79	0.79	0.78	231

Penjelasan:

Classification Report digunakan untuk menampilkan beberapa metrik evaluasi seperti precision, recall, F1-score, dan accuracy.

Precision menunjukkan tingkat ketepatan prediksi positif, recall menunjukkan kemampuan model dalam menemukan data positif, sedangkan F1-score merupakan harmonic mean antara precision dan recall.

12. ROC Curve dan AUC Score



Penjelasan:

ROC Curve digunakan untuk mengevaluasi kemampuan model dalam membedakan kelas positif dan negatif pada berbagai nilai threshold.

Sedangkan AUC Score digunakan untuk mengukur luas area di bawah kurva ROC. Semakin tinggi nilai AUC, maka semakin baik kemampuan model dalam melakukan klasifikasi data.

TUGAS PRAKTIKUM:

1. Scaling Dataset

```
PRAKTIKUM 3 / Tugas Logistic Regression.ipynb | import pandas as pd
Generate + Code + Markdown | Colab | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
at to Generate Code (Ctrl+I) | born as sns
[1] ✓ 3.8s
```

```
[2]
df = pd.read_csv('https://raw.githubusercontent.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/main/PRAKTIKUM3203/diabetes.csv')
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
[3]
print("Info Dataset:")
df.info()

print("Statistik deskriptif sebelum scaling:")
df.describe().round(2)
```

```
[4]
Info Dataset:
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Pregnancies            768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure          768 non-null   int64
3   SkinThickness          768 non-null   int64
4   Insulin                768 non-null   int64
5   BMI                   768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                   768 non-null   int64
8   Outcome                768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
Statistik deskriptif sebelum scaling:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00	768.00
mean	3.85	120.89	69.11	20.54	79.80	31.99	0.47	33.24	0.35
std	3.37	31.97	19.36	15.95	115.24	7.88	0.33	11.76	0.48
min	0.00	0.00	0.00	0.00	0.00	0.00	0.08	21.00	0.00
25%	1.00	99.00	62.00	0.00	0.00	27.30	0.24	24.00	0.00
50%	3.00	117.00	72.00	23.00	30.50	32.00	0.37	29.00	0.00
75%	6.00	140.25	80.00	32.00	127.25	36.60	0.63	41.00	1.00
max	17.00	199.00	122.00	99.00	846.00	67.10	2.42	81.00	1.00

```
X = df.drop(columns=['Outcome'])
y = df['Outcome']

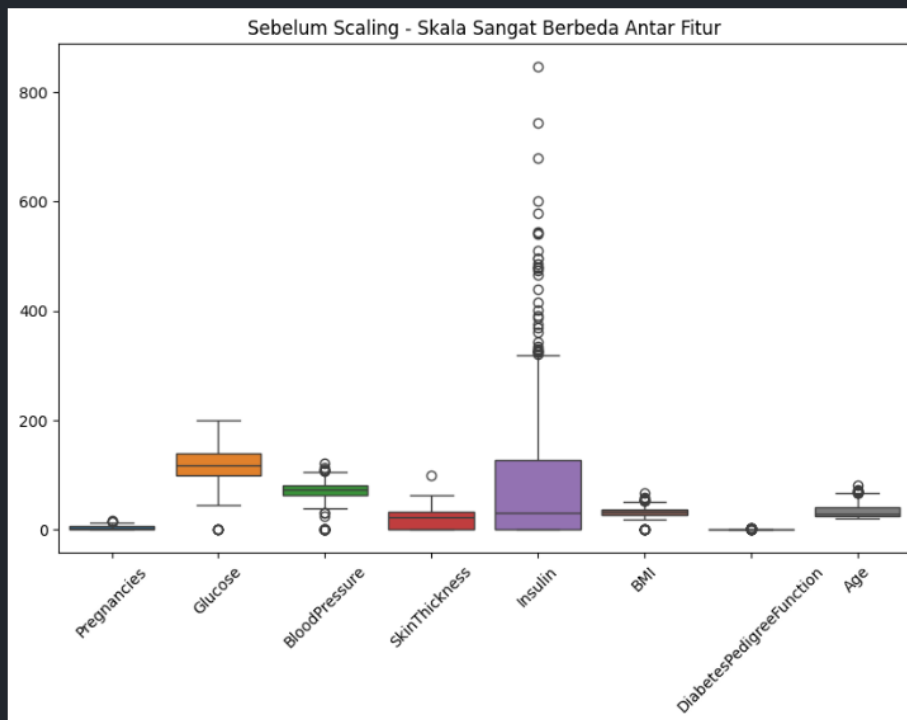
print("Fitur yang akan di-scaling:")
print(X.columns.tolist())
```

[4]

```
... Fitur yang akan di-scaling:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']
```

```
plt.figure(figsize=(10,6))
sns.boxplot(data=X)
plt.title("Sebelum Scaling - Skala Sangat Berbeda Antar Fitur")
plt.xticks(rotation=45)
plt.show()
```

[5]



```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

print("5 baris pertama setelah scaling:")
X_scaled_df.head().round(3)
```

[6]

```
... 5 baris pertama setelah scaling:
```

```
... 
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	0.640	0.848	0.150	0.907	-0.693	0.204	0.468	1.426
1	-0.845	-1.123	-0.161	0.531	-0.693	-0.684	-0.365	-0.191
2	1.234	1.944	-0.264	-1.288	-0.693	-1.103	0.604	-0.106
3	-0.845	-0.998	-0.161	0.155	0.123	-0.494	-0.921	-1.042
4	-1.142	0.504	-1.505	0.907	0.766	1.410	5.485	-0.020

```

print("Statistik SEBELUM scaling:")
print(X.describe().round(3))

print("\nStatistik SETELAH scaling:")
print(X_scaled_df.describe().round(3))

```

[7]

```

... Statistik SEBELUM scaling:
      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI \
count      768.000  768.000      768.000      768.000  768.000  768.000
mean         3.845  120.895        69.105       20.536   79.799   31.993
std          3.370   31.973        19.356       15.952  115.244    7.884
min           0.000    0.000         0.000        0.000    0.000    0.000
25%           1.000   99.000        62.000        0.000    0.000   27.300
50%           3.000  117.000        72.000       23.000   30.500   32.000
75%           6.000  140.250        80.000       32.000  127.250   36.600
max          17.000  199.000       122.000       99.000  846.000   67.100

      DiabetesPedigreeFunction   Age
count      768.000  768.000
mean         0.472   33.241
std          0.331   11.760
min          0.078   21.000
25%          0.244   24.000
50%          0.372   29.000
75%          0.626   41.000
max          2.420   81.000

Statistik SETELAH scaling:
      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI \
count      768.000  768.000      768.000      768.000  768.000  768.000
mean        -0.000  -0.000         0.000        0.000  -0.000    0.000
...
25%          -0.689  -0.786
50%          -0.300  -0.361
75%          0.466   0.660
max           5.884   4.064

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Penjelasan:

Pada tahap ini dilakukan proses scaling menggunakan StandardScaler agar seluruh fitur memiliki skala yang seragam. Scaling diperlukan karena Logistic Regression sensitif terhadap perbedaan skala antar fitur. Sebelum scaling, beberapa fitur memiliki rentang nilai yang jauh berbeda sehingga dapat memengaruhi proses pelatihan model. Setelah scaling, nilai fitur berubah menjadi bentuk standar dengan rata-rata mendekati nol dan simpangan baku mendekati satu.

2. Membagi Data Menjadi Data Latih dan Data Uji

```
#Split Data
from sklearn.model_selection import train_test_split

X_train_scaled, X_test_scaled, y_train, y_test = train_test_split(
    X_scaled_df, y, test_size=0.2, random_state=42, stratify=y
)

X_train, X_test, _, _ = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

[8]

```
... Train size: (614, 8)
     Test size: (154, 8)
```

Penjelasan:

Dataset dibagi menjadi data latih dan data uji menggunakan `train_test_split` dengan proporsi 80% data latih dan 20% data uji. Pembagian dilakukan pada dua versi data, yaitu data asli tanpa scaling dan data yang sudah dilakukan scaling. Hal ini bertujuan untuk membandingkan performa Logistic Regression pada kedua kondisi tersebut.

3. Fungsi Evaluasi Model dengan Threshold

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

def evaluate_with_threshold(y_true, y_proba, threshold=0.5):
    y_pred = (y_proba >= threshold).astype(int)

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    print(f"Threshold = {threshold:.2f}")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision : {prec:.4f}")
    print(f"Recall : {rec:.4f}")
    print(f"F1-score : {f1:.4f}")
    print(f"Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
    print(f"-"*40)

    return {'threshold' : threshold, 'acc' : acc, 'prec' : prec, 'rec' : rec, 'f1' : f1}
```

Penjelasan:

Fungsi ini dibuat untuk mengevaluasi model pada berbagai nilai threshold. Prediksi probabilitas dari model diubah menjadi label kelas berdasarkan batas threshold yang ditentukan. Setelah itu dihitung beberapa metrik evaluasi, yaitu accuracy, precision, recall, dan F1-score. Fungsi ini memudahkan analisis pengaruh threshold terhadap performa model klasifikasi.

4. Logistic Regression Tanpa Scaling

```
from sklearn.linear_model import LogisticRegression

model_no_scale = LogisticRegression(max_iter=1000)
model_no_scale.fit(X_train, y_train)

proba_no_scale = model_no_scale.predict_proba(X_test)[: , 1]

print("Tanpa Scaling:")
results_no_scale = []
for th in [0.3, 0.4, 0.5, 0.6, 0.7]:
    res = evaluate_with_threshold(y_test, proba_no_scale, th)
    results_no_scale.append(res)
```

[10]

```
... Tanpa Scaling:
Threshold = 0.30
Accuracy : 0.7338
Precision : 0.5867
Recall : 0.8148
F1-score : 0.6822
Confusion Matrix:
[[69 31]
 [10 44]]
-----
Threshold = 0.40
Accuracy : 0.7273
Precision : 0.6071
Recall : 0.6296
F1-score : 0.6182
Confusion Matrix:
[[78 22]
 [20 34]]
-----
Threshold = 0.50
Accuracy : 0.7143
Precision : 0.6087
Recall : 0.5185
F1-score : 0.5600
Confusion Matrix:
...
Confusion Matrix:
[[92  8]
 [34 20]]
-----
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Penjelasan:

Pada tahap ini model Logistic Regression dilatih menggunakan data yang belum dilakukan scaling. Setelah model dilatih, dilakukan prediksi probabilitas terhadap data uji. Hasil probabilitas tersebut kemudian dievaluasi pada beberapa nilai threshold yang berbeda untuk melihat perubahan performa model.

5. Logistic Regression Dengan Scaling

```
model_scaled = LogisticRegression(max_iter=1000)
model_scaled.fit(X_train_scaled, y_train)

proba_scaled = model_scaled.predict_proba(X_test_scaled)[: , 1]

print("Dengan Scaling:")
results_scaled = []
for th in [0.3, 0.4, 0.5, 0.6, 0.7]:
    res = evaluate_with_threshold(y_test, proba_scaled, th)
    results_scaled.append(res)
```

[11]

```
... Dengan Scaling:
Threshold = 0.30
Accuracy : 0.7403
Precision : 0.5946
Recall : 0.8148
F1-score : 0.6875
Confusion Matrix:
[[70 30]
 [10 44]]
```

```
-----
Threshold = 0.40
Accuracy : 0.7338
Precision : 0.6140
Recall : 0.6481
F1-score : 0.6306
Confusion Matrix:
[[78 22]
 [19 35]]
```

```
-----
Threshold = 0.50
Accuracy : 0.7143
Precision : 0.6087
Recall : 0.5185
F1-score : 0.5600
Confusion Matrix:
...
Confusion Matrix:
[[92  8]
 [34 20]]
-----
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

Penjelasan:

Model Logistic Regression juga dilatih menggunakan data yang sudah dilakukan scaling. Prediksi probabilitas kemudian dievaluasi pada beberapa nilai threshold yang sama seperti pada model tanpa scaling. Tahap ini dilakukan agar dapat dibandingkan apakah scaling memberikan pengaruh terhadap performa model.

6. Perbandingan Hasil Model

```
import pandas as pd

df_compare = pd.DataFrame({
    'Threshold': [r['threshold'] for r in results_no_scale],
    'Acc (No Scale)': [r['acc'] for r in results_no_scale],
    'F1 (No Scale)': [r['f1'] for r in results_no_scale],
    'Acc (Scaled)': [r['acc'] for r in results_scaled],
    'F1 (Scaled)': [r['f1'] for r in results_scaled],
})

print("Perbandingan Kinerja:")
print(df_compare.round(4))

# Kalau mau lihat yang mana lebih baik di threshold 0.5
print("\nPada threshold default 0.5:")
print(df_compare[df_compare['Threshold'] == 0.5])
```

[12]

```
... Perbandingan Kinerja:
      Threshold  Acc (No Scale)  F1 (No Scale)  Acc (Scaled)  F1 (Scaled)
0           0.3           0.7338           0.6822           0.7403           0.6875
1           0.4           0.7273           0.6182           0.7338           0.6306
2           0.5           0.7143           0.5600           0.7143           0.5600
3           0.6           0.7338           0.5393           0.7338           0.5393
4           0.7           0.7273           0.4878           0.7273           0.4878

Pada threshold default 0.5:
      Threshold  Acc (No Scale)  F1 (No Scale)  Acc (Scaled)  F1 (Scaled)
2           0.5           0.714286           0.56           0.714286           0.56
```

Penjelasan:

Hasil evaluasi dari model dengan scaling dan tanpa scaling disusun ke dalam bentuk tabel agar lebih mudah dibandingkan. Dari tabel ini dapat dilihat pengaruh scaling terhadap accuracy dan F1-score pada tiap threshold. Selain itu, threshold default 0.5 juga ditampilkan secara khusus karena biasanya menjadi acuan utama dalam klasifikasi biner.

7. ROC Curve dan AUC Score

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Hitung ROC curve dan AUC untuk No Scale
fpr_ns, tpr_ns, thresholds_ns = roc_curve(y_test, proba_no_scale)
roc_auc_ns = auc(fpr_ns, tpr_ns)

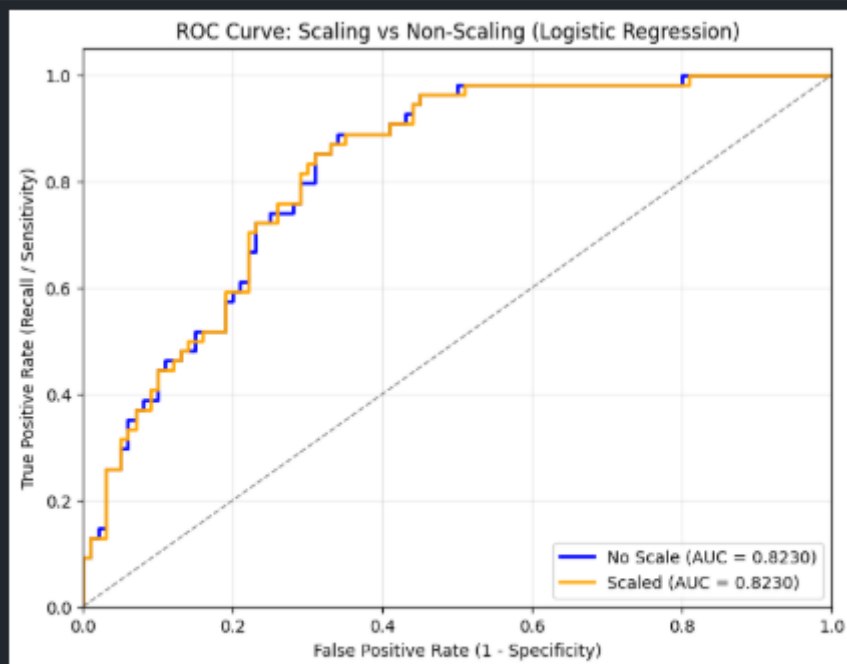
# Hitung ROC curve dan AUC untuk Scaled
fpr_s, tpr_s, thresholds_s = roc_curve(y_test, proba_scaled)
roc_auc_s = auc(fpr_s, tpr_s)

# Plot ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr_ns, tpr_ns, color='blue', lw=2, label=f'No Scale (AUC = {roc_auc_ns:.4f})')
plt.plot(fpr_s, tpr_s, color='orange', lw=2, label=f'Scaled (AUC = {roc_auc_s:.4f})')

# Garis diagonal (random classifier)
plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Recall / Sensitivity)')
plt.title('ROC Curve: Scaling vs Non-Scaling (Logistic Regression)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.show()

# Print AUC saja kalau mau cepat lihat
print(f"AUC No Scale : {roc_auc_ns:.4f}")
print(f"AUC Scaled : {roc_auc_s:.4f}")
```



```
AUC No Scale : 0.8230
AUC Scaled : 0.8230
```

Penjelasan:

ROC Curve digunakan untuk melihat kemampuan model dalam membedakan kelas positif dan negatif pada berbagai threshold. AUC Score digunakan untuk mengukur luas area di bawah kurva ROC. Semakin besar nilai AUC, maka semakin baik kemampuan model dalam melakukan klasifikasi. Dengan membandingkan kurva ROC dari model tanpa scaling dan dengan scaling, dapat diketahui model mana yang memiliki performa lebih baik secara keseluruhan.

8. Analisis Hasil

Penjelasan:

Berdasarkan percobaan yang dilakukan, Logistic Regression menunjukkan bahwa proses scaling berpengaruh terhadap kestabilan dan performa model. Pada data yang memiliki perbedaan skala fitur yang cukup besar, scaling membantu model bekerja lebih optimal. Selain itu, perubahan threshold juga memengaruhi hasil klasifikasi. Threshold yang lebih rendah cenderung meningkatkan recall, sedangkan threshold yang lebih tinggi cenderung meningkatkan precision. Oleh karena itu, pemilihan threshold perlu disesuaikan dengan tujuan klasifikasi yang ingin dicapai.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Logistic Regression merupakan metode klasifikasi yang efektif untuk melakukan prediksi pada kasus binary classification seperti klasifikasi diabetes. Model mampu mempelajari hubungan antara fitur-fitur kesehatan dengan kemungkinan pasien mengalami diabetes melalui fungsi sigmoid yang menghasilkan probabilitas klasifikasi.

Hasil evaluasi menggunakan confusion matrix, accuracy, precision, recall, F1-score, ROC Curve, dan AUC Score menunjukkan bahwa model memiliki performa klasifikasi yang cukup baik. Selain itu, proses scaling menggunakan StandardScaler membantu meningkatkan kestabilan model karena seluruh fitur memiliki skala yang seragam.

Percobaan terhadap beberapa nilai threshold juga menunjukkan bahwa perubahan threshold memengaruhi performa model, terutama pada nilai precision dan recall. Threshold yang lebih rendah cenderung meningkatkan recall, sedangkan threshold yang lebih tinggi meningkatkan precision. Oleh karena itu, pemilihan threshold yang tepat sangat penting agar model dapat bekerja sesuai kebutuhan klasifikasi yang diinginkan.